Lecture Notes: Avoid the Avoidable Inefficiencies

Course: High Performance Computing 1 (DSAI + SDIC)
Lecturer: Luca Tornatore
Academic Year: 2025/2026

Premise

Before we dive into the proper optimisation techniques — cache-aware traversals, branch elimination, pipeline-friendly instruction scheduling — there is a preliminary "hygiene" to take care of. It is not glamorous, and it does not require any deep understanding of the hardware; it requires, instead, that you develop the habit of *thinking about what your code actually does* at every line, and that you stop relying on the hope that the compiler will fix your carelessness.

The point of this lecture is not to turn you into micro-optimisation maniacs. It is to calibrate your instinct: to make you recognise, at first glance, the kind of waste that should never appear in production-grade scientific code.
Kind of: learn to avoid crude grammatical errors before worrying about good style.

---

1. Algorithmic Cleanliness: Expensive Functions

When implementing mathematical formulas, it is natural — and in fact tempting — to translate the mathematical notation directly into code.
After all,
$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$
translates literally into a call to `sqrt()` wrapping three calls to `pow()`.
It compiles, it runs, it gives the right answer. The problem is that not all operations are created equal (I mean, have the same cost), and the difference can amount to one or two orders of magnitude.

To appreciate this concretely, look up the instruction latency tables in the Intel Intrinsics Guide or in Agner Fog's instruction tables (https://www.agner.org/optimize/instruction_tables.pdf): on a modern x86-64 core, an `addsd` (scalar double add) or `mulsd` (scalar double multiply) has a latency of about 3–5 cycles and a throughput of one per cycle. An `sqrtsd` (scalar double square root) costs roughly 13–19 cycles depending on the microarchitecture. An `fdiv` (floating-point division) is in the same ballpark, around 13–20 cycles. These are hardware instructions — they are as fast as the silicon can make them.

Now, `pow()`, `exp()`, `log()`, `sin()`, `cos()` are a different story entirely. These are not single hardware instructions; they are *library functions* that implement transcendental approximations using polynomial expansions, range-reduction algorithms, and careful rounding. A call to the standard `pow()` in glibc can cost anywhere from 50 to over 200 cycles, depending on the arguments and the implementation. It must handle edge cases mandated by the IEEE 754 standard and the C standard: what happens when the base is negative? When the exponent is NaN? When the result overflows? It must set `errno` on certain error conditions. All of this machinery is invoked every single time you call `pow()` — even if you are merely computing x^2.

The `pow()` case in detail

Consider the innocent-looking expression `pow(dx, 2.0)`. When the exponent is a floating-point literal like `2.0`, and hence known at compile-time, the compiler sees a call to the generic `pow(double, double)` function. Under *strict standard compliance*, it cannot simply replace this with `dx * dx`, because the two are not semantically identical: `pow()` with a floating-point exponent must handle the full generality of $x^y = e^{y \ln x}$, including edge cases (negative zero, infinities, NaN propagation) and setting `errno` on overflow — none of which a simple multiplication does.

Now, all major compilers — GCC, Clang/LLVM, and Intel's compilers — *do* attempt to recognise `pow(x, n)` where the exponent is a small integer known at compile time, and replace the call with an appropriate multiplication chain. GCC, for instance, has a built-in `__builtin_powi` that it can substitute.

But here is the **critical detail**: this substitution only happens reliably when the exponent is of integer type, and when certain flags or built-in recognition applies.
With `pow(dx, 2.0)` — where `2.0` is a `double` — the compiler must first prove that `2.0` is exactly an integer, and then further satisfy itself that the substitution does not alter the observable behaviour of the program (the `errno` issue, the special-value handling).
In practice, GCC at `-O2` will often replace `pow(x, 2.0)` with a multiplication, but this is *not guaranteed by any standard*, and the behaviour varies across compiler versions and flags. With `-fno-math-errno` the substitution becomes far more reliable, because the compiler no longer needs to preserve the error-reporting semantics.

The practical advice is simple.
For **small integer exponents** (2, 3, 4, perhaps 5), write the multiplication explicitly. `dx * dx` is unambiguous, portable, and costs a few cycles. `pow(dx, 2.0)` might or might not be optimised away, depending on your compiler, your flags, and the phase of the moon.
For **large or non-integer exponents**, `pow()` is of course the right tool. The same reasoning applies to `sqrt()`: if you only need the distance for a comparison (e.g., "is this point within radius R?"), compare the *squared* distance against R^2 and avoid the square root entirely. Only call `sqrt()` when you genuinely need the actual distance value for subsequent computation.

Floating-point division

Division is another case worth internalising. On modern x86, an `fdiv` or `divsd` instruction has a latency of 13–20 cycles and — critically — a throughput that is much worse than multiplication (often one every 4–6 cycles, compared to one multiplication per cycle). If you are dividing by the same constant repeatedly inside a loop — say, `i / Ng` for each grid index — you should precompute the reciprocal `Ng_inv = 1.0 / Ng` and multiply by it. This is such a common pattern that the compiler flag `-freciprocal-math` exists precisely to let the compiler do it for you; but again, writing it explicitly costs you nothing and guarantees the optimisation.

---

2. Loop Hoisting: Saying What You Mean

Consider the nested loop from the slides, where for each point p we scan a 3D grid of $N_g \times N_g \times N_g$ cells. In the naive version, the innermost loop body computes:

```
dx = x[p] - (double)i * Ng_inv - half_size;
dy = y[p] - (double)j * Ng_inv - half_size;
dz = z[p] - (double)k * Ng_inv - half_size;
```

The expression for `dx` depends only on the index `i` from the outermost grid loop; it does not change as `j` and `k` vary. Yet it is being recomputed N_g^2 times for every value of `i` — once for each `(j, k)` pair. Likewise, the expression for `dy` depends only on `j`, but it is recomputed N_g times for every `(i, j)` pair as `k` varies. Only `dz` genuinely belongs in the innermost loop.

Extracting these computations to the appropriate loop level — "hoisting" them — eliminates a vast amount of redundant work. Before hoisting, the total number of evaluations of the grid-coordinate expressions across all three dimensions is $3N_g^3$ (each of the three coordinates is evaluated in the innermost loop). After hoisting, it becomes $N_g + N_g^2 + N_g^3$. For a grid of, say, $N_g = 100$, this means going from 3×10^6 evaluations to roughly 1.01×10^6 — and each "evaluation" involves a cast, a multiplication, and a subtraction. That is not nothing.

The hoisted code reads:

```
for (int i = 0; i < Ng; i++) {
    double dx  = x[p] - (double)i * Ng_inv - half_size;
    double dx2 = dx * dx;

    for (int j = 0; j < Ng; j++) {
        double dy       = y[p] - (double)j * Ng_inv - half_size;
        double dist2_xy = dx2 + dy * dy;

        for (int k = 0; k < Ng; k++) {
            double dz    = z[p] - (double)k * Ng_inv - half_size;
            double dist2 = dist2_xy + dz * dz;

            if (dist2 < Rmax2)
                /* do something with sqrt(dist2) */;
        }
    }
}
```

One can go further: if the grid coordinates $i \cdot N_g^{-1} + \text{half_size}$ are used identically for all three dimensions, precomputing them into a lookup array `ijk[Ng]` before the particle loop saves even the per-particle re-evaluation:

```
double ijk[Ng];
for (int i = 0; i < Ng; i++)
    ijk[i] = (double)i * Ng_inv + half_size;
```

A modern compiler at `-O3` will, in many cases, perform **loop-invariant code motion** (LICM) and hoist these computations on its own.
So why bother doing it manually? Two reasons.
First, if the compiler suspects that memory aliasing might occur — for instance, if the grid-coordinate arrays and the particle arrays could overlap — it will conservatively refuse to hoist. Writing the hoisted code explicitly removes any ambiguity.
Second, and more importantly, the act of hoisting forces you to *understand the dependency structure of your*

loop. It makes the computational complexity visible in the code structure itself. The hoisted version does not just run faster; it *communicates more clearly* what work is actually necessary.
In performance-critical scientific code, that clarity is not a luxury — it is the first line of defence against the kind of **invisible inefficiency that accumulates across thousands of lines** and eventually makes a simulation twice as slow as it needs to be.

---

3. Variable Scope and Register Pressure

In older C89-style code, it was common practice to declare all variables at the beginning of a function, in a monolithic block. You would see something like:

```
void compute_forces(int n, double *x, double *y, ...) {
    int i, j, k;
    double dx, dy, dz, dist2, dist2_xy, temp, Rmax2;
    double fx, fy, fz;
    /* ... 200 lines of code using these variables
       in various unrelated contexts ... */
}
```

This is an awful habit, and abandoning it in favour of declaring variables at their point of first use (as C99 and later permit) is worthwhile for two distinct reasons. One is very solid; the other is more nuanced and worth discussing honestly.

The solid reason: human clarity and correctness

When you declare a variable at the top of a 200-line function and then reuse it in three different loop nests for three different purposes, you are creating a maintenance minefield. The variable name `temp` tells nobody anything; the fact that it was last assigned in some distant `for` loop is invisible at the point of its next use. Worse, you may inadvertently create a data dependency where none should exist: if `temp` was supposed to be "fresh" at the start of a new loop but still carries a value from the previous one, you have a bug — and a subtle one, because the code *might* produce correct results for most inputs and only break in corner cases.

Declaring variables in the tightest scope that contains their actual use eliminates this class of bugs by construction. If `dx2` only exists inside the `for(i...)` block, there is no possibility of accidentally reading a stale value of `dx2` elsewhere.

The more nuanced reason: helping the compiler

The compiler's register allocator works by analysing the "live range" of each variable — the interval between the point where it receives a value and the last point where that value is read. When many variables have overlapping live ranges, the allocator may run out of hardware registers and be forced to "spill" some of them to the stack. Spilling means writing a register's value to memory and reloading it later, which adds latency and memory traffic.

At first glance, declaring variables in tight scopes shortens their live ranges and should therefore reduce register pressure. And this reasoning is correct — in principle.

However, at optimisation levels $\geq$ `-O2`, modern compilers (GCC, LLVM) perform their analysis on an intermediate representation in SSA (Static Single Assignment) form, where the original declaration position of a variable is essentially irrelevant. The compiler decomposes your variables into "virtual registers" based on their actual use-def chains, splits live ranges where beneficial, and reuses physical registers across non-overlapping lifetimes — regardless of where you wrote the `double dx;` in your source code. A variable declared at the top of the function but used only inside one loop will, at `-O2`, likely get the same register treatment as one declared inside that loop.

So does scope matter for performance?
At `-O0` and `-O1`, where the compiler performs less aggressive analysis, tighter scopes can genuinely help.
At `-O2` and above, the effect is usually negligible. But there is a subtler point: giving the compiler clean, unambiguous code — where the programmer's *intent* is manifest in the structure — makes the compiler's job easier and more robust.
It is the difference between handing someone a well-organised filing cabinet and handing them a pile of papers with the instruction "you figure it out." A good compiler can handle the pile, but why make it harder?

What about stack cache locality? This argument — that keeping variables with related lifetimes close in declaration helps keep them close on the stack and therefore close in cache — is sometimes made, but it is largely a non-issue when the stack frame involves a handful of scalar variables, totalling perhaps 64–128 bytes. This fits within a single cache line or two (we'll see that), and it will almost certainly be hot in L1 cache throughout the loop's execution. Stack locality becomes a genuine concern only in very large stack in functions with thousands of lines (that should be avoided, in any case), and/or deeply recursive code with large stack frames..

The bottom line: declare variables at the point of first use, with the tightest scope possible. Do it primarily for the sake of code clarity, maintainability, and correctness. Do it secondarily because it *may* help the compiler, and it certainly cannot hurt. But do not expect miracles from it at `-O2`.

---

4. Don't Repeat Unnecessary Work

The `strlen()` example from the slides deserves a brief discussion in the notes, because it illustrates a pattern that is far more common than one might think.

```
char *find_char_in_string(char *string, char c) {
    int i = 0;
    while (i < strlen(string))
        if (string[i] == c) break;
        else i++;
    if (i < strlen(string)) return &string[i];
    else return NULL;
}
```

The function `strlen()` is not a constant-time operation. It scans the string from the beginning until it finds the null terminator `'\0'`, which makes it $O(n)$. Calling it in the loop condition means that *every iteration* of the `while` loop triggers a full rescan of the entire string — turning what should be a linear-time search into a quadratic-time one, $O(n^2)$. For a string of length 10, nobody will notice. For a string of length 10^6 (which is not unusual when parsing data files or processing genomic sequences), the difference is between 10^6 and 10^{12} character comparisons.

One might hope that the compiler would recognise that `string` does not change and hoist the `strlen()` call. But it cannot: `string` is received as a pointer, and the compiler has no way to prove that the memory it points to is not modified between iterations — perhaps by a signal handler, or by another thread, or by some other pointer aliased to the same region. The compiler must be conservative and assume the worst.

The fix is trivial: compute the length once, store it, and use the stored value. Better yet, rewrite the function to walk the string with a pointer and check for `'\0'` inline, avoiding `strlen()` altogether:

```
char *find_char_in_string(char *string, char c) {
    char *pos = string;
    while (*pos != '\0' && *pos != c)
        pos++;
    return (*pos == '\0') ? NULL : pos;
}
```

This version is $O(n)$, reads each character exactly once, uses no function calls, and is trivially correct. It also, incidentally, gives the compiler a much easier time: the loop body is tiny, branch-free (apart from the loop condition), and amenable to vectorisation on architectures that support it.

---

5. Avoid Unnecessary Memory References

Consider this simple reduction loop:

```
void reduce_vector(int n, double *array, double *sum) {
    for (int i = 0; i < n; i++)
        *sum += array[i];
}
```

The code accumulates into `*sum`, which means that at each iteration the compiler must load the current value of `*sum` from memory, add `array[i]`, and store the result back to the memory location pointed to by `sum`. When compiled at `-O1`, this produces an inner loop with three memory operations: a load from `*sum`, an add from `array[i]`, and a store back to `*sum`.

Why doesn't the compiler simply keep the running total in a register? Because of memory aliasing. The compiler cannot prove that `sum` and `array` do not point to overlapping memory regions. If, say, `sum == &array[3]`, then writing to `*sum` would change `array[3]`, and the result of the next `array[i]` load would depend on the write. Holding the accumulator in a register and writing it back only once at the end would produce a *different* result in this aliased scenario. The compiler must preserve the semantics you wrote, even if they are not what you intended.

The fix is to use a local accumulator:

```
void reduce_vector(int n, double *array, double *sum) {
    double temp = 0;
    for (int i = 0; i < n; i++)
        temp += array[i];
    *sum = temp;
}
```

Now `temp` is a local variable — it has no address (unless you take one), it cannot alias anything, and the compiler is free to keep it in a register for the duration of the loop. The inner loop at `-O1` shrinks to a single `addsd` and a pointer increment. This is also a textbook case where the `restrict` qualifier would help: declaring the function as `reduce_vector(int n, double *restrict array, double *restrict sum)` tells the compiler that the two pointers do not alias, enabling the register optimisation without the temporary variable. Both approaches are valid; the local accumulator has the advantage of being explicit and compiler-independent.

---

6. The Illusion of Associativity: Floating-Point Arithmetic

In mathematics, integer and real-number addition is associative: $(a + b) + c = a + (b + c)$. In floating-point (FP) arithmetic on a digital computer, **this is false**.

The reason is fundamental: a floating-point number represents a real number using a fixed number of significant digits (the mantissa or significand). IEEE 754 double precision gives you 52 explicit bits of mantissa, which corresponds to about 15–16 decimal digits. When two numbers of vastly different magnitude are added, the smaller one may be partially or entirely "absorbed" — its contribution lost to rounding.

A concrete example

Suppose we work in a decimal floating-point system with 3 digits of precision. Then:

$$1.00 + 0.01 = 1.01 \quad (\text{fits in 3 digits, exact})$$

$$1.00 + 0.001 = 1.00 \quad (\text{would need 4 digits: } 1.001 \to \text{rounded to } 1.00)$$

Now consider summing eleven numbers: ten copies of 0.001 and one copy of 1.00. If we sum the small numbers first, we get 0.01 (which fits in 3 digits), and then $0.01 + 1.00 = 1.01$. If instead we start with 1.00 and add 0.001 ten times, *each addition* produces 1.00 (because $1.00 + 0.001 = 1.00$ in 3-digit arithmetic), and the final result is 1.00. Same values, different order, different answer. This is not a pathology; it is an inherent property of finite-precision arithmetic.

The consequence for compilers is severe: because the order of floating-point operations affects the result, a compiler is **not permitted** to rearrange your FP expressions, even if a mathematically equivalent rearrangement would be faster.
It must faithfully execute the operations in the order you wrote them.
This means, for instance, that the compiler cannot split a serial summation loop into multiple independent partial sums (which would expose ILP and enable SIMD vectorisation), because doing so would change the order of additions.
It means the compiler cannot replace `a / b * c / b` with `a * c / (b * b)` or `a * c * (1.0 / b) * (1.0 / b)`, because the intermediate roundings would differ.
In short, strict IEEE compliance prevents many of the most effective code transformations that the compiler could otherwise perform.

This is also a source of headaches in parallel computing, a source of indeterminism and possible “chaotic behavirou”, and it is worth stressing. If you split a summation across multiple threads or MPI ranks and then combine the partial results, the final answer will depend on how the work is distributed and in what order the partial sums are recombined. This is not a bug — it is an intrinsic consequence of finite-precision arithmetic, and it means that parallel floating-point computations are, in a sense, inherently non-deterministic at the last few bits of precision.

Advanced mitigation techniques (a simple example: the Kahan summation) can shallow the problem but not eliminate it entirely; a deeper discussion is available in the materials on the course's GitHub repository (`kahan_summation`).

Relaxing the Rules: Compiler Flags and Their Trade-Offs

To achieve peak performance in floating-point-heavy code, we sometimes need to we often must re-organize wisely our loops, and/or grant the compiler the explicit permission to violate strict IEEE 754 compliance. The relevant GCC flags (with broadly equivalent options in Clang and Intel compilers) fall into a natural hierarchy, from the most targeted to the most aggressive.

(1) `-fassociative-math`

This is perhaps the single most performance-relevant flag for scientific computing. It tells the compiler that it may treat floating-point addition and multiplication as associative — that is, it may reorder, regroup, and redistribute FP operations as if they were real-number operations on the blackboard.

What does this unlock? Consider a simple reduction loop:

```
double sum = 0;
for (int i = 0; i < n; i++)
    sum += a[i];
```

Without `-fassociative-math`, this is a serial dependency chain: each addition depends on the result of the previous one, giving a throughput of one addition per ~3–5 cycles (the latency of `addsd`). With `-fassociative-math`, the compiler is free to split this into, say, four independent partial sums:

```
double s0 = 0, s1 = 0, s2 = 0, s3 = 0;
for (int i = 0; i < n; i += 4) {
    s0 += a[i];   s1 += a[i+1];
    s2 += a[i+2]; s3 += a[i+3];
}
sum = s0 + s1 + s2 + s3;
```

This breaks the dependency chain, exposes ILP (all four additions are independent and can be pipelined), and sets the stage for SIMD vectorisation (the four additions can become a single `vaddpd` on a 256-bit register). The potential speedup is enormous — easily 4× to 8× on the inner loop.

The cost is that the result may differ at the last few bits of precision, because the order of additions has changed. **For most scientific applications, this is perfectly acceptable**; the roundoff differences are far smaller than the physical uncertainties in the data. But **for applications where bit-reproducibility matters** (checksumming, regression testing, certain financial calculations), it is not.

(2) `-fno-math-errno` and `-funsafe-math-optimizations`

These address two separate but related constraints:

`-fno-math-errno` tells the compiler that math library functions (like `sqrt`, `pow`, `exp`, `log`) need not set the global variable `errno` on error conditions such as domain errors or overflow. In the C standard, calling `sqrt(-1.0)` must set `errno` to `EDOM`. Ensuring this requires the library implementation to check the arguments, branch on special cases, and write to a global variable — all of which prevent the compiler from

inlining a fast hardware instruction in place of the library call. With `-fno-math-errno`, the compiler can freely replace `sqrt(x)` with the hardware `sqrtsd` instruction (which returns NaN for negative inputs but does not set `errno`), and can inline and simplify other math functions similarly.

This is one of the safest flags to use. Most scientific code never checks `errno` after math library calls; if your code relies on checking `errno`, then of course that option is out-of-scope.

`-funsafe-math-optimizations` is a broader permission that enables several transformations: it allows the compiler to assume that arguments to math functions are within their valid domains, to use less precise but faster approximations for certain functions, and (in combination with `-fassociative-math`, which it implies) to rearrange expressions more freely.
The "unsafe" in the name is perhaps overly alarming; the real danger is not that it produces garbage, but that it may change results by a few ULPs (Units in the Last Place) compared to the strictly conformant computation.

(3) `-freciprocal-math`

This flag allows the compiler to replace a/b with $a \times (1/b)$, and more generally to share reciprocals across multiple divisions by the same value. Since FP division is much slower than FP multiplication (13–20 cycles vs. 3–5 cycles), this can be a significant win when dividing by the same variable in multiple expressions. The trade-off is that $a \times (1/b)$ may produce a result that differs from a/b by one ULP, because the intermediate reciprocal $1/b$ itself incurs a rounding.

This is the flag equivalent of the manual optimisation of precomputing `Ng_inv = 1.0 / Ng` that we discussed earlier. If you already do the precomputation manually, the flag has no additional effect. If you do not, the flag automates the transformation.

(4) `-ffast-math` — use with extreme caution, or not at all

This is the "enable everything" umbrella flag. It turns on `-fassociative-math`, `-fno-math-errno`, `-funsafe-math-optimizations`, `-freciprocal-math`, `-ffinite-math-only`, `-fno-signed-zeros`, `-fno-trapping-math`, and more. The most dangerous of these sub-flags are:

- **`-ffinite-math-only`**: tells the compiler that `NaN` and `Inf` will never occur. This means that any code that checks for `NaN` (e.g., `if (isnan(x))`) or `Inf` (e.g., `if (isinf(x))`) will be silently optimised away, because the compiler is entitled to assume these conditions can never be true. If your simulation uses such checks to detect and handle physical singularities (gravitational collapse, particle overlap, division by zero distances), **this flag will silently disable your safety checks**. The code will compile, run, and produce subtly wrong results — or crash in unpredictable ways.
- **`-fno-signed-zeros`**: treats $+0$ and -0 as identical. This is usually harmless, but can affect codes that rely on the sign of zero to determine the branch of a multi-valued function (e.g., `atan2`).

The advice is: **never use `-ffast-math` as a blanket flag unless you have exhaustively verified that your code does not depend on any of the behaviours it disables.** Instead, enable the individual sub-flags that you actually need. In most scientific computing contexts, `-fassociative-math -fno-math-errno` gives you the lion's share of the performance benefit with a well-understood and acceptable trade-off. Adding `-freciprocal-math` is usually safe as well. Going beyond that requires case-by-case analysis.

For a thorough and well-written discussion of the subtleties, two resources are particularly recommended:

- Simon Byrne's notes on `-ffast-math`: https://simonbyrne.github.io/notes/fastmath/
- Krister Walfridsson's blog post: https://kristerw.github.io/2021/10/19/fast-math/

Key Messages

The techniques in this lecture are not sophisticated, and they are not optional. They are the *minimum standard* for professional scientific code. A compiler is a powerful tool, but it operates under strict rules — rules that often prevent it from performing optimisations that are, from a physicist's perspective, obviously correct. The compiler cannot know that you will never pass `NaN` to `sqrt()`. It cannot know that your two pointers do not alias. It cannot know that your string does not change length mid-scan. It cannot know that the order of your additions does not matter for your application.

Your job is to make these things explicit: in the code structure, in the variable scoping, in the choice of compiler flags. Write the code you mean. Do not write something sloppy and hope the compiler will figure out what you *intended* — because in many cases, it will not, and in some cases, it *must not*.

Understand the tools. Understand the trade-offs. And write clean code from the start.